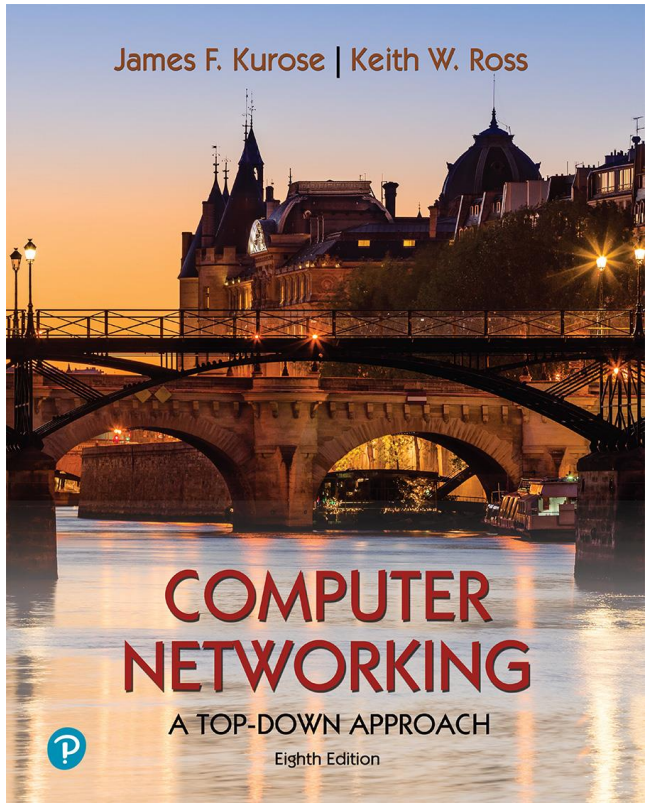
Chương 2
Tầng ứng dụng



Computer Networking: A Top-Down Approach

8th edition n
Jim Kurose, Keith Ross
Pearson, 2020

© All material copyright 1996-2012
J.F Kurose and K.W. Ross, All Rights Reserved

Chương 2: Tầng ứng dụng

Mục tiêu:

- Khái niệm, các vấn đề cài đặt giao thức tầng ứng dụng
  - Các mô hình dịch vụ tầng giao vận
  - Mô hình khách-chủ (client-server)
  - Mô hình điểm-điểm (peer-to-peer)
- Nghiên cứu một số giao thức tầng ứng dụng
  - HTTP
  - FTP
  - SMTP / POP3 / IMAP
  - DNS
- Tạo một ứng dụng mạng
  - socket API

Chương 2: Nội dung

2.1 Nguyên lý của ứng dụng mạng

2.2 Web và HTTP

2.3 FTP

2.4 Thư điện tử

- SMTP, POP3, IMAP

2.5 DNS

2.6 Ứng dụng P2P

2.7 Lập trình socket với UDP và TCP

Một số ứng dụng mạng

- Mạng xã hội
- Thư điện tử (e-mail)
- web
- Tin nhắn văn bản (text messaging)
- Chia sẻ file P2P
- Trò chơi nhiều người trên mạng
- streaming video (YouTube, Hulu, Netflix)
- Điện thoại Internet (voice over IP) (ví dụ Skype)
- Hội thảo video thời gian thực (ví dụ: zoom, meet)
- Các ứng dụng tìm kiếm
- Truy nhập từ xa (remote login)
- …

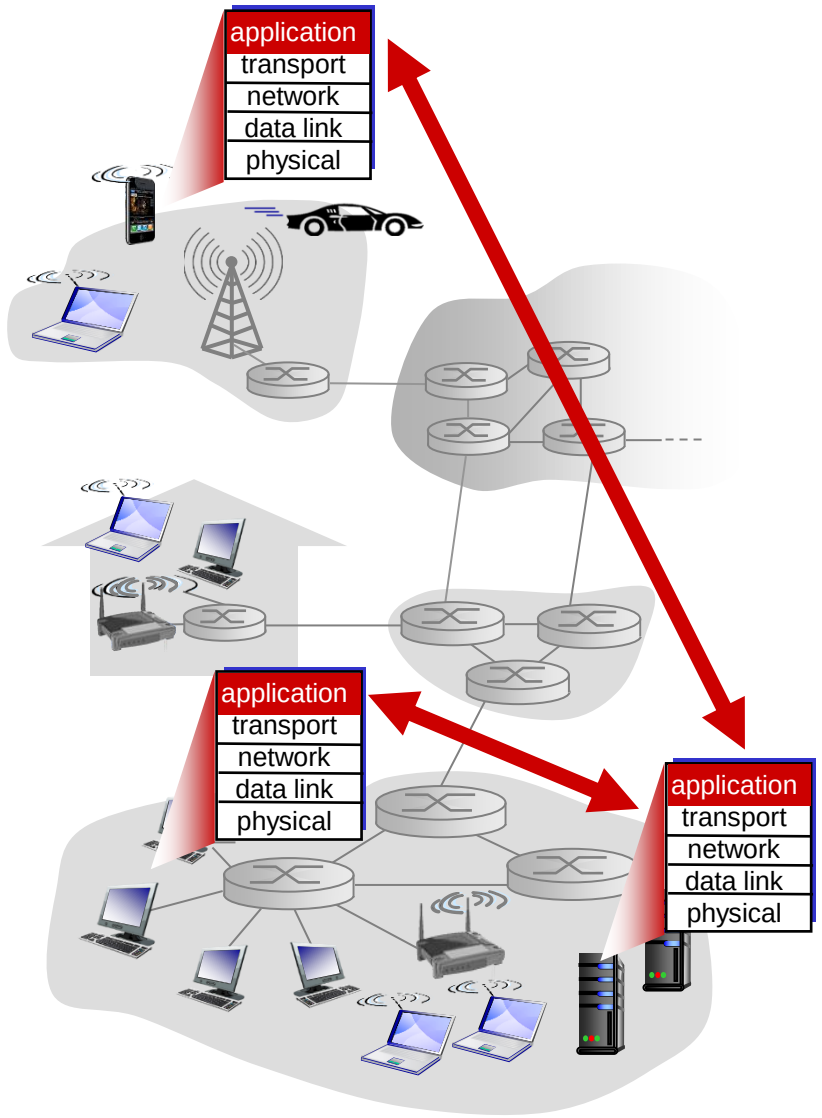
Tạo một ứng dụng mạng

Viết chương trình để:

- Chạy trên các *hệ thống đầu cuối* khác nhau
- Truyền thông qua mạng
- Ví dụ: phần mềm máy chủ web (web server) truyền thông với phần mềm trình duyệt (browser software)

Không cần viết chương trình ứng dụng cho các thiết bị trong phần lõi của mạng

- Các thiết bị trong phần lõi của mạng không chạy các ứng dụng người dùng.
- Các ứng dụng chạy trên thiết bị đầu cuối cho phép phát triển và phổ biến ứng dụng một cách nhanh chóng.

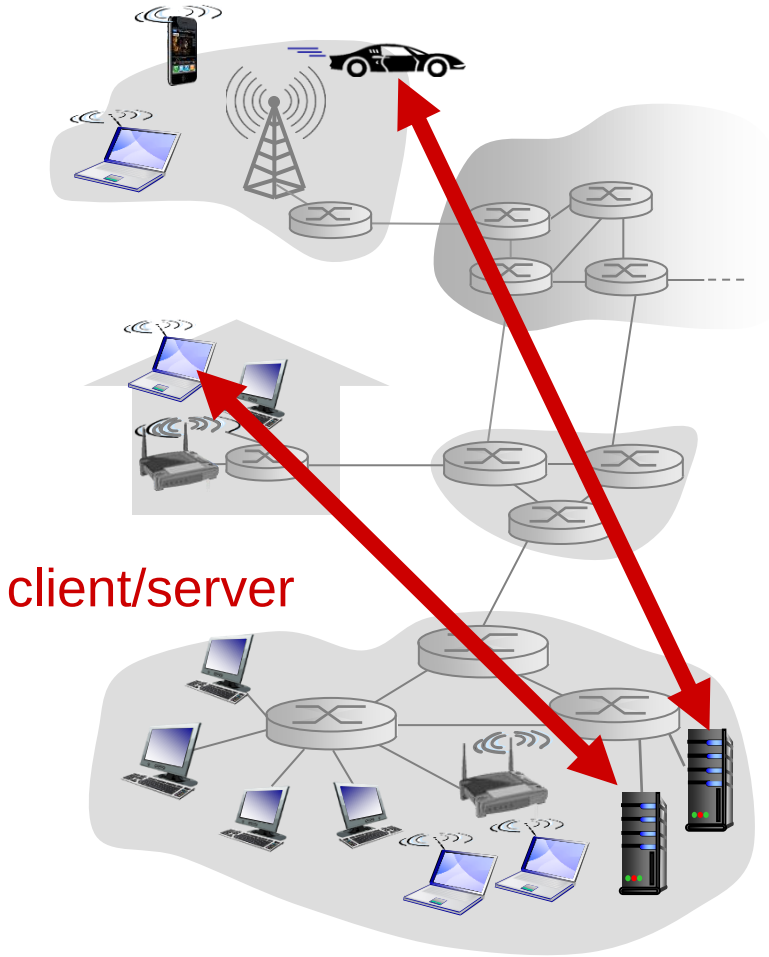


Kiến trúc của ứng dụng

Các ứng dụng có thể có kiến trúc dạng:

- Client-server (khách-chủ)
- Peer-to-peer (P2P, ngang hàng)

Kiến trúc client-server



server:

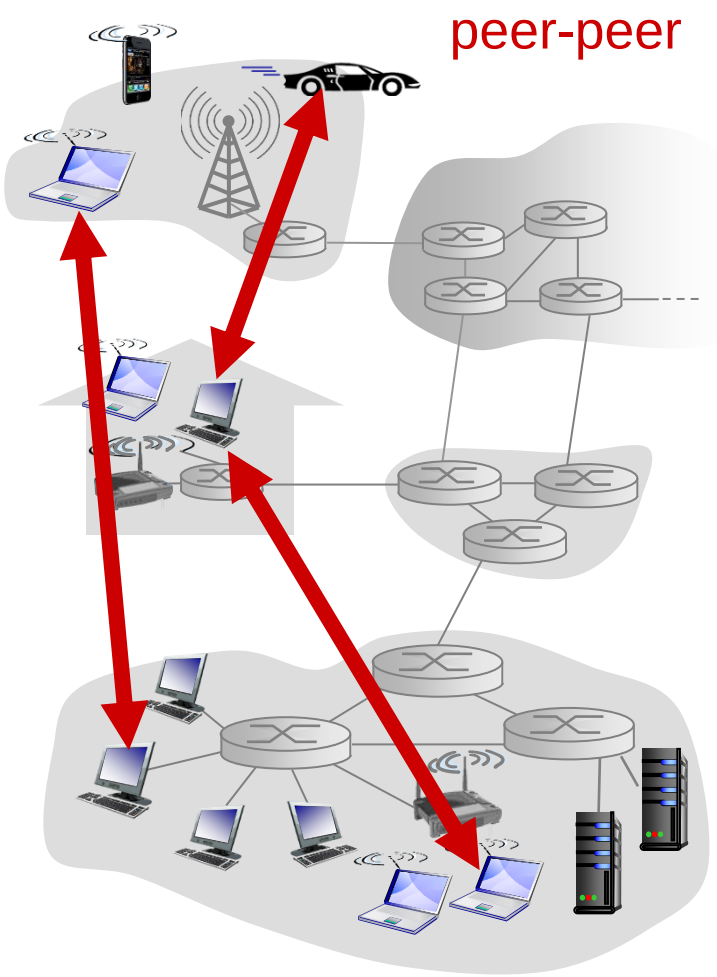
- Là host luôn hoạt động
- Có địa chỉ IP cố định
- Các trung tâm dữ liệu

client:

- Truyền thông với server
- Có thể được kết nối liên tục vào mạng hoặc không
- Có thể có địa chỉ IP thay đổi
- Không truyền thông trực tiếp với client khác

Kiến trúc P2P

- *Không có* server luôn hoạt động
- Các hệ thống đầu cuối (peer) truyền thông trực tiếp với nhau
- Mỗi peer yêu cầu dịch vụ từ một peer nào đó, và cung cấp dịch vụ lại cho các peer khác.
  - *Có khả năng tự mở rộng* – peer mới mang lại khả năng dịch vụ mới, cũng như có những yêu cầu dịch vụ mới.
- Các peer không kết nối liên tục và có thể thay đổi địa chỉ IP
  - Quản lý phức tạp



Tiến trình truyền thông

Tiến trình: chương trình chạy trên một host

- Trên cùng một host, hai tiến trình truyền thông với nhau qua truyền thông tiến trình nội bộ (được xác định bởi hệ điều hành)
- Các tiến trình trên các host khác nhau truyền thông với nhau bằng cách trao đổi các thông điệp

client, server

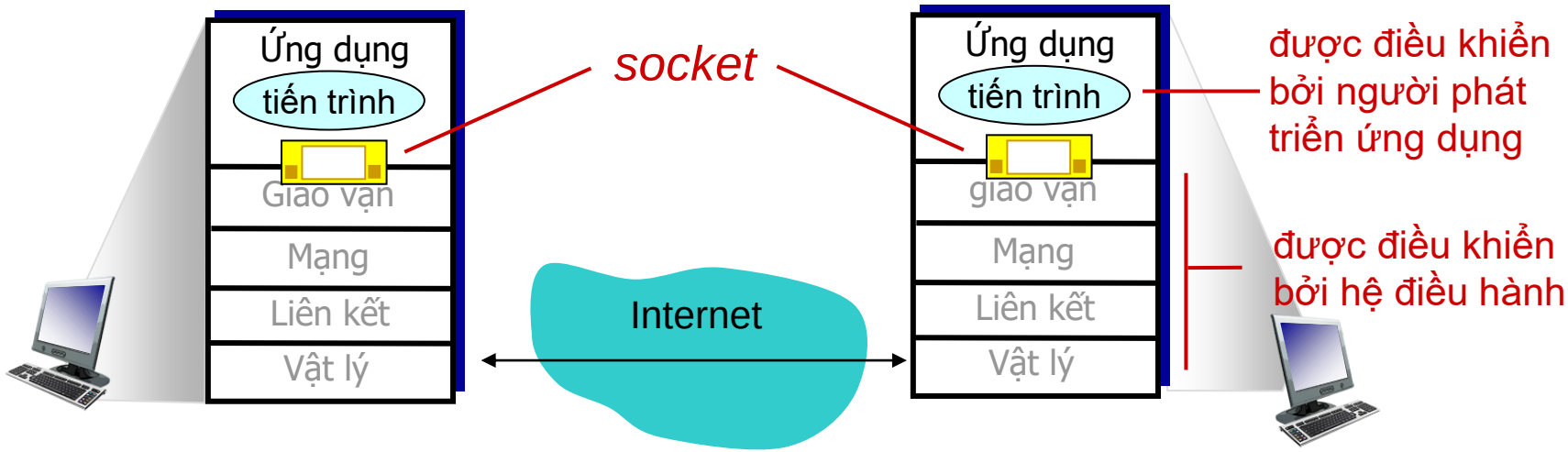
Tiến trình client: tiến trình khởi tạo truyền thông

Tiến trình server: tiến trình chờ được tiếp xúc

- Lưu ý: ứng dụng theo kiến trúc P2P có cả tiến trình client & tiến trình server

Socket

- Các tiến trình gửi/nhận các thông điệp đến/từ socket của nó
- Socket tương tự như một cửa vào/ra
  - Tiến trình gửi đẩy thông điệp ra bên ngoài cửa
  - Tiến trình gửi dựa trên cơ sở hạ tầng tầng giao vận ở phía bên kia cửa để truyền thông điệp đến socket của tiến trình nhận



Định địa chỉ cho tiến trình

- Để nhận các thông điệp, tiến trình phải có *định danh (identifier)*
- Thiết bị host phải có địa chỉ IP 32-bit duy nhất
- *Câu hỏi:* Địa chỉ IP của host mà trên đó tiến trình chạy có đủ để xác định định danh của tiến trình không?
  - *Trả lời:* Không, *do nhiều* tiến trình có thể cùng chạy trên một host.

- *Định danh* bao gồm cả địa chỉ IP và số hiệu cổng được kết hợp với tiến tình trên host.
- Ví dụ các số hiệu cổng:
  - HTTP server: 80
  - mail server: 25
- Để gửi thông điệp HTTP tới web server gaia.cs.umass.edu:
  - Địa chỉ IP: 128.119.245.12
  - Số hiệu cổng: 80

Các giao thức tầng ứng dụng

- Các loại thông điệp được trao đổi
  - Ví dụ: thông điệp yêu cầu (request), thông điệp đáp ứng (response)
- Cú pháp của thông điệp:
  - Các trường trong thông điệp & mô tả
- Ngữ nghĩa của thông điệp
  - Ý nghĩa thông tin của các trường
- Quy tắc về thời gian và cách thức các tiến trình gửi và đáp ứng thông điệp

Các giao thức mở (công khai):

- Được định nghĩa trong RFCs
- Cho phép tương tác
- Ví dụ: HTTP, SMTP

Các giao thức riêng (độc quyền):

- Ví dụ: Skype, Zoom

Một ứng dụng cần dịch vụ giao vận nào?

Toàn vẹn dữ liệu

- Một số ứng dụng (ví dụ: truyền file, web) yêu cầu truyền tin cậy 100%
- Các ứng dụng khác (ví dụ: audio) có thể chịu một số mất mát

Thời gian thực

- Một số ứng dụng (ví dụ: điện thoại Internet, trò chơi tương tác) yêu cầu độ trễ thấp mới "hiệu quả"

Thông lượng

- Một số ứng dụng (ví dụ: đa phương tiện) yêu cầu thông lượng tối thiểu để đảm bảo "hiệu quả".
- Các ứng dụng khác ("các ứng dụng mềm dẻo") có thể dùng băng thông nào cũng được

An toàn bảo mật

- Mã hóa, đảm bảo toàn vẹn dữ liệu, …

Một số yêu cầu dịch vụ giao vận với các ứng dụng phổ biến

Ứng dụng	Mất dữ liệu	Thông lượng	Thời gian thực
Truyền file	không	mềm dẻo	không
Thư điện tử	không	mềm dẻo	không
Web	không	mềm dẻo	không
Audio/video thời gian thực	chịu lỗi	audio: 5kbps-1Mbps video:10kbps-5Mbps	có, 100 msec
Lưu trữ audio/video	chịu lỗi	như trên	có, vài giây
Trò chơi tương tác	chịu lỗi	một vài kbps	có, 100 msec
Tin nhắn nhanh	không	mềm dẻo	có và không

Các dịch vụ giao thức tầng giao vận của Internet

Dịch vụ TCP:

- *Truyền tin cậy* giữa tiến trình gửi và tiến trình nhận
- *Điều khiển luồng:* bên gửi không lấn át bên nhận
- *Điều khiển tắc nghẽn:* điều tiết bên gửi khi mạng bị quá tải
- *Không cung cấp:* thời gian thực, đảm bảo băng thông tối thiểu, an toàn bảo mật
- *Hướng kết nối:* yêu cầu thiết lập kết nối giữa tiến trình client và tiến trình server

Dịch vụ UDP:

- *Truyền dữ liệu không tin cậy* giữa tiến trình gửi và tiến trình nhận
- *Không cung cấp:* truyền tin cậy, điều khiển luồng, điều khiển tắc nghẽn, thời gian thực, đảm bảo băng thông, an toàn bảo mật, hoặc thiết lập kết nối

Hỏi: Tại sao lại dùng cả hai dịch vụ? Dùng UDP để làm gì?

Các ứng dụng Internet: các giao thức tầng ứng dụng và giao vận

Ứng dụng	Giao thức tầng ứng dụng	Giao thức tầng giao vận
Thư điện tử	SMTP [RFC 5321]	TCP
Truy nhập từ xa	Telnet [RFC 854]	TCP
Web	HTTP [RFC 7230, 9110]	TCP
Truyền/tải file	FTP [RFC 959]	TCP
Streaming multimedia	HTTP (ví dụ: YouTube), RTP [RFC 1889]	TCP hoặc UDP
Điện thoại Internet	SIP, RTP, giao thức độc quyền (ví dụ: Skype)	TCP hoặc UDP

Bảo mật TCP

TCP & UDP

- Không mã hóa
- Mật khẩu dạng bản gốc được gửi vào socket để truyền trên Internet theo dạng bản gốc

SSL (Secure Sockets Layer)/TLS

- Hỗ trợ kết nối TCP được mã hóa
- Toàn vẹn dữ liệu
- Xác thực thiết bị đầu cuối

SSL/TLS là bảo mật tại tầng ứng dụng

- Ứng dụng dùng các thư viện của SSL/TLS để "gọi" tới TCP

Socket API của SSL

- Mật khẩu dạng bản gốc được gửi vào socket để truyền trên Internet theo dạng đã được mã hóa

Chương 2: Nội dung

2.1 Nguyên lý của ứng dụng mạng

2.2 Web và HTTP

2.3 FTP

2.4 Thư điện tử

- SMTP, POP3, IMAP

2.5 DNS

2.6 Ứng dụng P2P

2.7 Lập trình socket với UDP và TCP

Web và HTTP

Một số thuật ngữ:

- *Trang web* bao gồm một số *đối tượng (object) và chúng có thể được lưu ở các máy chỉ web khác nhau*
- Đối tượng có thể là tệp HTML, ảnh JPEG, Java applet, tệp audio,…
- Trang web bao gồm *tệp HTML cơ bản* chứa *một số đối tượng được tham chiếu*
- Mỗi đối tượng được định địa chỉ bởi một *URL,* ví dụ:

`www.someschool.edu/someDept/pic.gif`

Tên host | Tên đường dẫn

Khái quát về HTTP

HTTP: hypertext transfer protocol

- Giao thức tầng ứng dụng của Web
- Mô hình client/server
  - *client:* trình duyệt (browser) yêu cầu, nhận (sử dụng giao thức HTTP), và "hiển thị" các đối tượng Web
  - *server:* Máy chủ web (web server) gửi (sử dụng giao thức HTTP) các đối tượng đáp ứng cho yêu cầu.



Khái quát về HTTP

Dùng TCP:

- Client khởi tạo kết nối TCP (tạo socket) tới server, cổng 80
- Server chấp nhận kết nối TCP từ client
- Thông điệp HTTP (thông điệp giao thức tầng ứng dụng) được trao đổi giữa trình duyệt (HTTP client) và máy chủ Web (HTTP server)
- Đóng kết nối TCP

HTTP là "không trạng thái"

- Server không lưu giữ thông tin về những yêu cầu trước đó của client

Vấn đề liên quan

Giao thức lưu giữ "trạng thái" khá phức tạp!

- Lịch sử quá khứ (trạng thái) cần phải được lưu giữ
- Nếu server/client bị sự cố, thì thông tin về "trạng thái" của chúng có thể không nhất quán, cần phải được điều chỉnh.

Kết nối HTTP: có 2 kiểu

HTTP không liên tục

- Chỉ có tối đa một đối tượng được gửi qua một kết nối TCP
  - Kết nối sau đó sẽ được đóng lại
- Việc tải về nhiều đối tượng sẽ yêu cầu nhiều kết nối

HTTP liên tục

- Nhiều đối tượng có thể được gửi qua một kết nối TCP duy nhất giữa client và server

HTTP không liên tục

Giả sử người dùng gõ URL:

`www.someSchool.edu/someDepartment/home.index`

(chứa văn bản, tham chiếu tới 10 ảnh jpeg)

1a. HTTP client khởi tạo kết nối TCP tới HTTP server (tiến trình) tại www.someSchool.edu trên cổng 80

1b. HTTP server tại host www.someSchool.edu, đang chờ kết nối TCP tại cổng 80, "chấp nhận" kết nối, thông báo cho client

2. HTTP client gửi *thông điệp HTTP yêu cầu* (chứa URL) vào trong socket kết nối TCP. Thông điệp chỉ ra là client muốn lấy đối tượng someDepartment/home.index

3. HTTP server nhận thông điệp yêu cầu, định dạng *thông điệp đáp ứng* chứa đối tượng được yêu cầu, và gửi thông điệp vào trong socket của nó

thời gian ↓

HTTP không liên tục

4. HTTP server đóng kết nối TCP.

5. HTTP client nhận thông điệp đáp ứng chứa tệp HTML, hiển thị HTML. Phân tích tệp HTML, tìm 10 đối tượng jpeg được tham chiếu

6. Các bước từ 1 đến 5 được lặp lại cho từng đối tượng trong 10 đối tượng jpeg

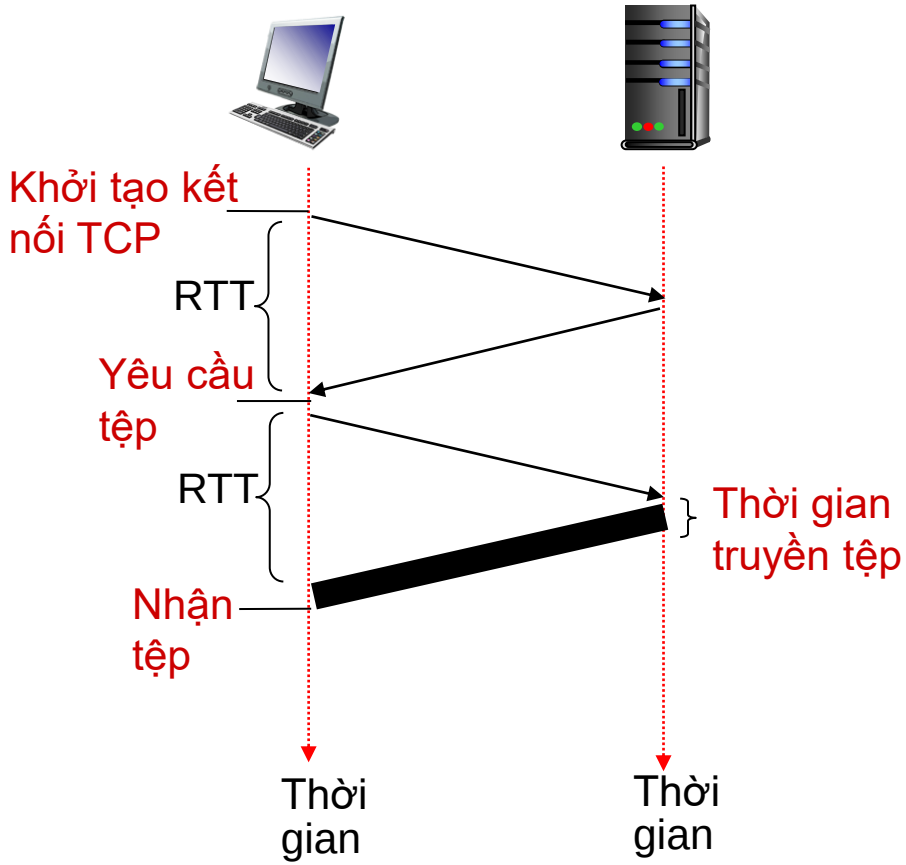
thời gian

HTTP không liên tục: thời gian đáp ứng

RTT (định nghĩa): thời gian để một gói tin nhỏ đi từ client đến server và quay lại.

Thời gian đáp ứng HTTP:

- Một RTT để khởi tạo kết nối TCP
- Một RTT để gửi HTTP yêu cầu và một vài byte HTTP đáp ứng được trả về
- Thời gian truyền tệp



Thời gian đáp ứng của HTTP không liên tục = 2RTT+ thời gian truyền tệp

HTTP liên tục (HTTP 1.1)

Vấn đề với HTTP không liên tục:

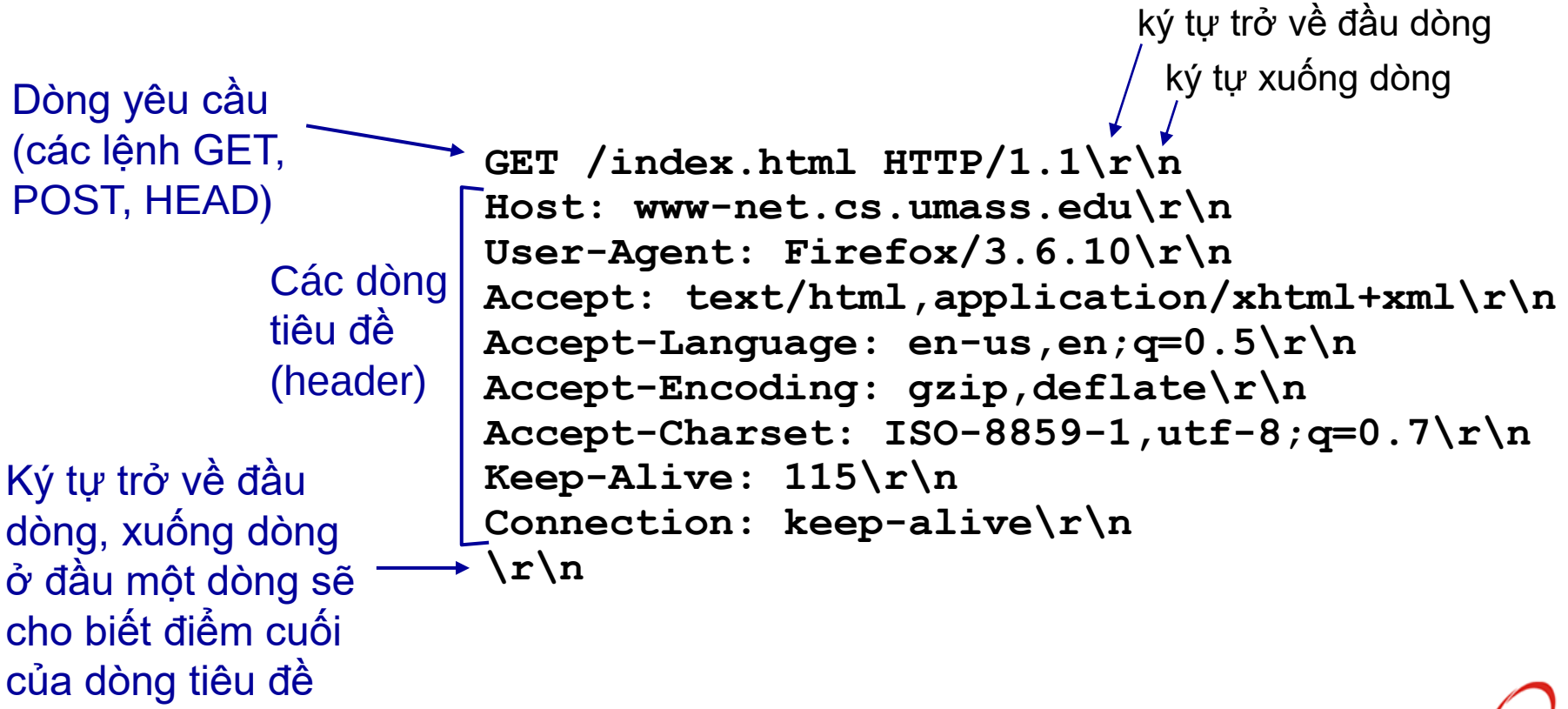
- Cần 2 RTT cho mỗi đối tượng
- Hệ điều hành liên quan đến *mỗi* kết nối TCP
- Các trình duyệt thường mở nhiều kết nối TCP song song để lấy các đối tượng được tham chiếu

HTTP liên tục:

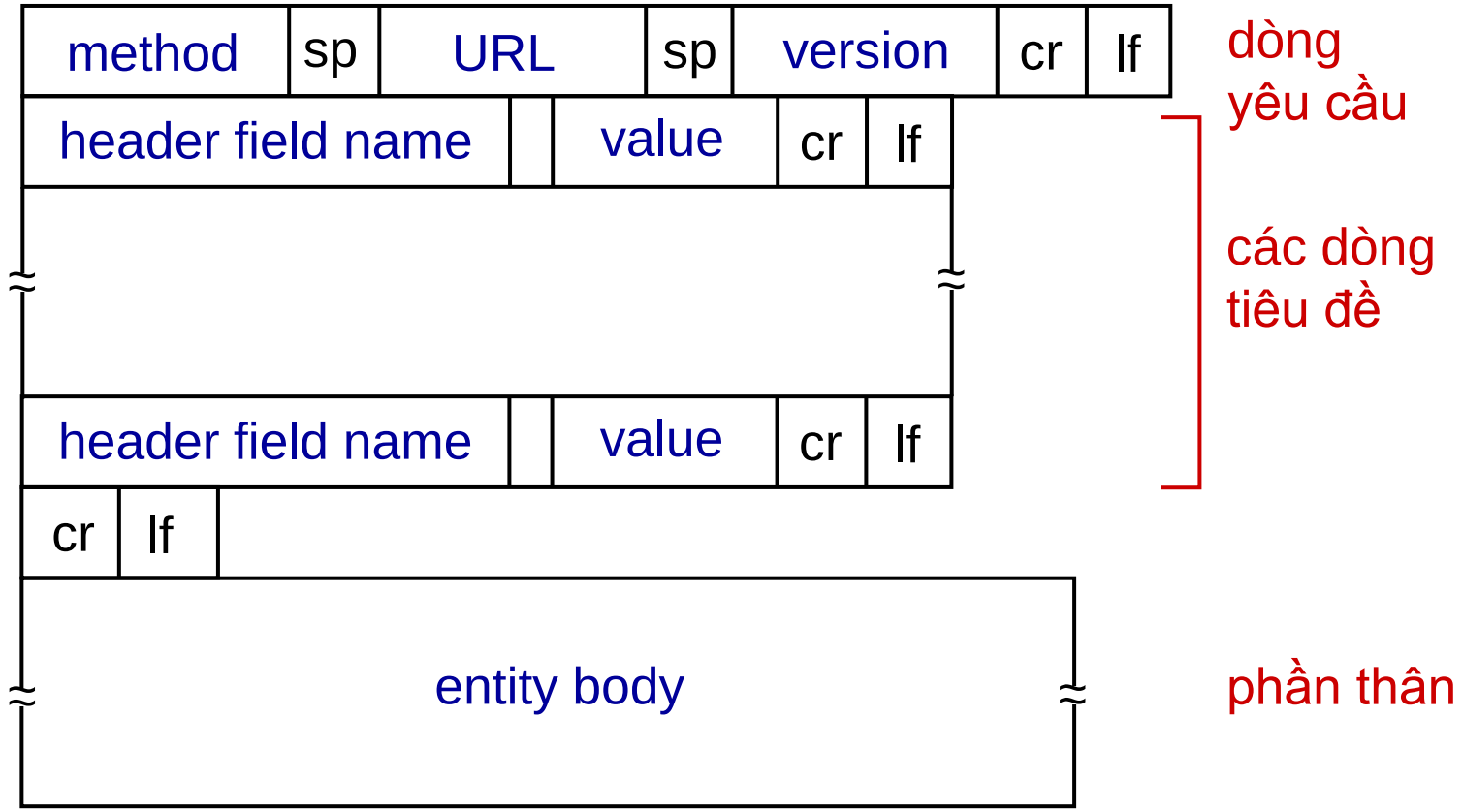
- Server để mở kết nối sau khi gửi đáp ứng
- Chuỗi các thông điệp HTTP tiếp theo giữa cùng client/server sẽ được gửi thông qua kết nối mở này
- Client gửi yêu cầu ngay sau khi gặp một đối tượng được tham chiếu
- Ít nhất là một RTT cho tất cả các đối tượng được tham chiếu

Thông điệp yêu cầu HTTP

- Có hai loại thông điệp HTTP: *yêu cầu và đáp ứng*
- Thông điệp HTTP yêu cầu:
  - ASCII (định dạng con người có thể đọc được)



Thông điệp HTTP yêu cầu: định dạng tổng quát



Các loại phương thức

HTTP/1.0:

- GET
- POST
- HEAD
  - Yêu cầu server dời đối tượng được yêu cầu ra khỏi đáp ứng

HTTP/1.1:

- GET, POST, HEAD
- PUT
  - Tải tệp lên trong phần thân thực thể tới đường dẫn được xác định trong trường URL
- DELETE
  - Xóa tệp được xác định trong trường URL

Các mã trạng thái đáp ứng HTTP

- Mã trạng thái xuất hiện ngay trong dòng đầu tiên của thông điệp đáp ứng từ server đến client.
- Ví dụ một số mã:

200 OK

- Yêu cầu thành công, đối tượng yêu cầu nằm ở phía sau thông điệp này

301 Moved Permanently

- Đối tượng yêu cầu đã bị di chuyển, vị trí mới được xác định ở phía sau thông điệp này (Location:)

400 Bad Request

- Server không hiểu thông điệp yêu cầu

404 Not Found

- Tài liệu yêu cầu không có trong server này

505 HTTP Version Not Supported

Trạng thái User-server: cookies

Nhiều trang Web sử dụng các cookies

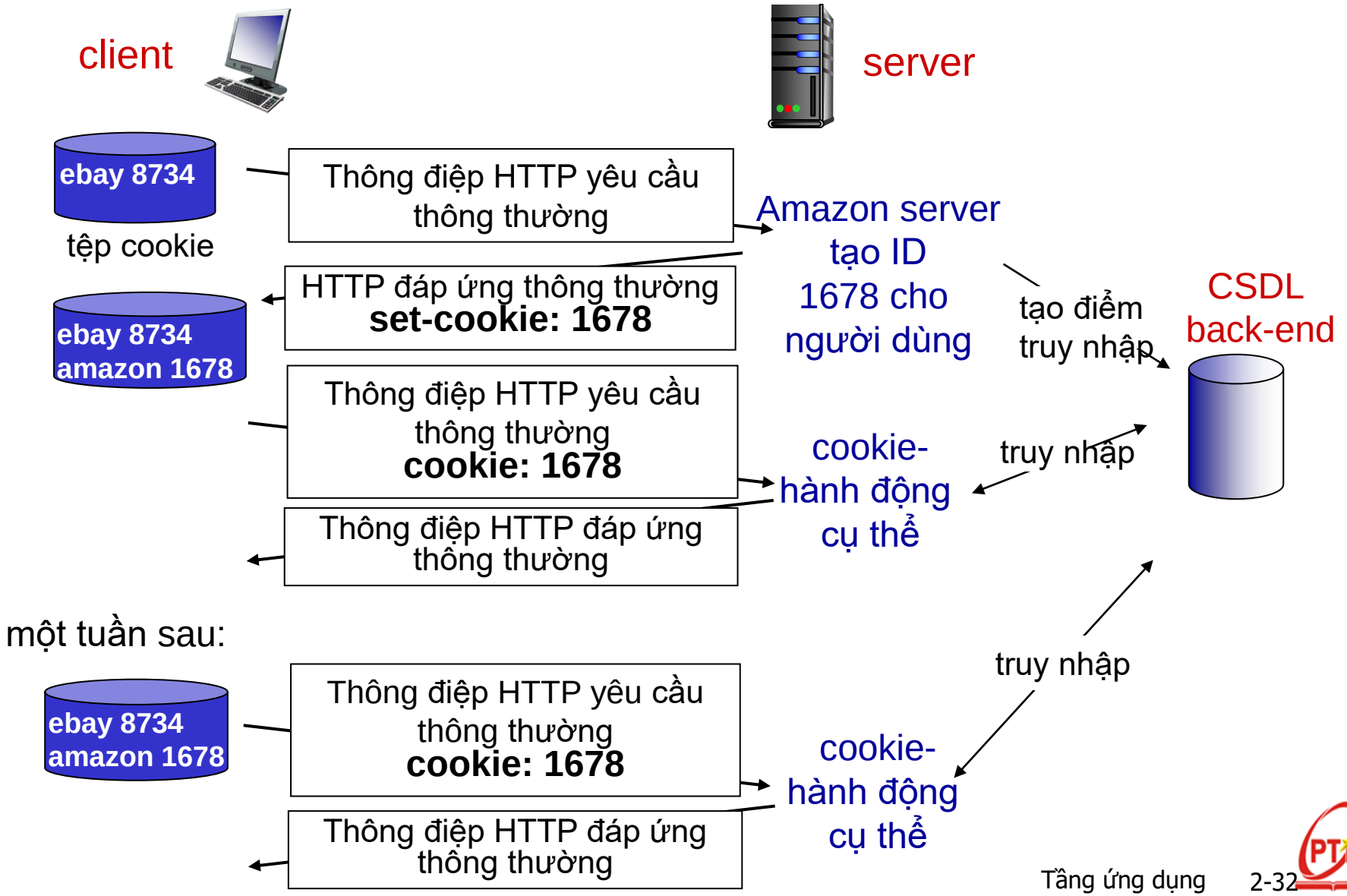
Bốn thành phần:

1) Dòng tiêu đề cookie của thông điệp HTTP *đáp ứng*
2) Dòng tiêu đề cookie trong thông điệp HTTP *yêu cầu* tiếp theo
3) Tệp cookie được lưu giữ trên host của người dùng, được quản lý bởi trình duyệt của người dùng.
4) Cơ sở dữ liệu back-end tại Web site

Ví dụ:

- Susan thường truy nhập Internet từ một PC
- Cô ấy vào một trang thương mại điện tử lần đầu tiên
- Khi yêu cầu HTTP khởi tạo đi đến trang, trang sẽ tạo ra:
  - Một ID duy nhất
  - Điểm truy nhập vào cơ sở dữ liệu back-end cho ID

Các cookie: lưu giữ "trạng thái" (tiếp)



Các cookie (tiếp)

Các cookie có thể được dùng cho những việc gì?

- Cấp phép
- Giỏ mua hàng
- Khuyến nghị
- Trạng thái phiên làm việc của người dùng (Web e-mail)

Ngoài ra

cookie và sự riêng tư:

- Cookie cho phép các site biết nhiều hơn về người dùng
- Người dùng có thể cung cấp tên và e-mail cho các site

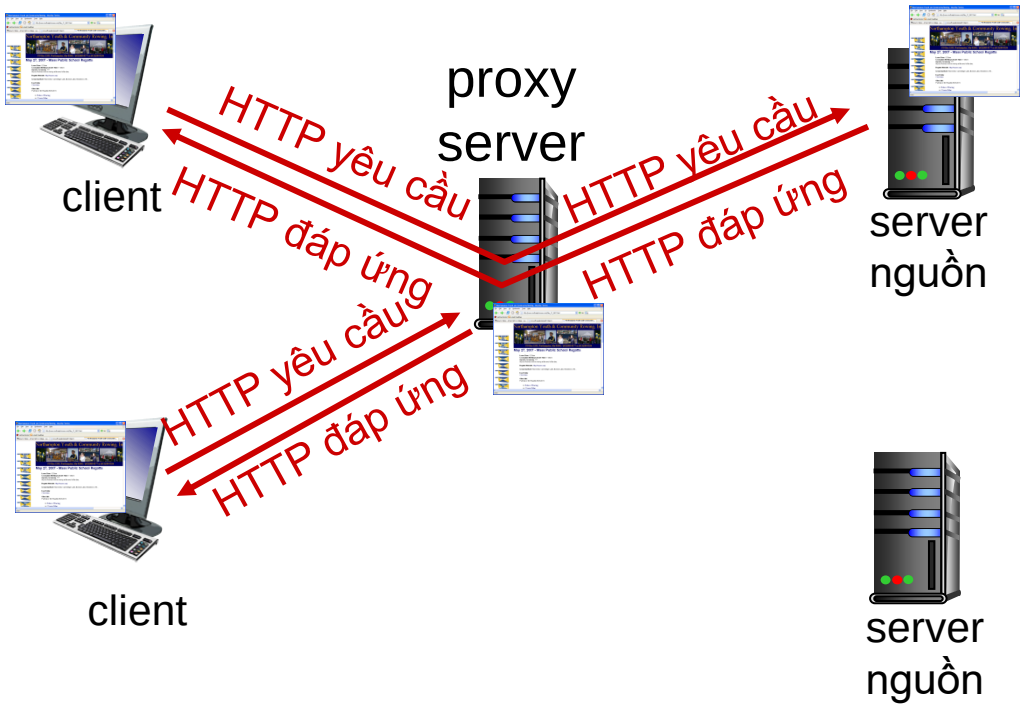
Làm cách nào để lưu giữ "trạng thái":

- Các điểm cuối giao thức: duy trì trạng thái tại bên gửi/bên nhận thông qua nhiều giao dịch
- Các cookie: các thông điệp HTTP mang trạng thái

Web caches (proxy server)

Mục tiêu: thỏa mãn yêu cầu của client mà không cần liên quan đến server nguồn

- Người dùng thiết lập trình duyệt: truy nhập Web qua vùng nhớ đệm (cache)
- Trình duyệt gửi tất cả các yêu cầu HTTP tới cache
  - Nếu đối tượng có trong cache: cache sẽ trả về đối tượng
  - Ngược lại, cache sẽ yêu cầu đối tượng từ server nguồn, sau đó sẽ trả đối tượng về cho client



Web caching

- Bộ nhớ cache hoạt động ở cả client và server
  - Server đáp ứng cho yêu cầu đầu tiên từ một client
  - Client gửi yêu cầu tới server gốc
- Cache thường được cài đặt bởi ISP (trường học, công ty, khu dân cư)

Tại sao lại cần Web caching?

- Giảm được thời gian đáp ứng cho yêu cầu của client
- Giảm được lưu lượng trên một liên kết truy nhập của tổ chức
- Nếu Internet thực hiện cache nhiều thì sẽ làm cho các nhà cung cấp nội dung "nghèo đi" phân phối nhiều nội dung đó (cũng tương tự như chia sẻ file P2P).

Chương 2: Nội dung

2.1 Nguyên lý của ứng dụng mạng
- Kiến trúc của ứng dụng
- Các yêu cầu của ứng dụng

2.2 Web và HTTP

2.3 FTP

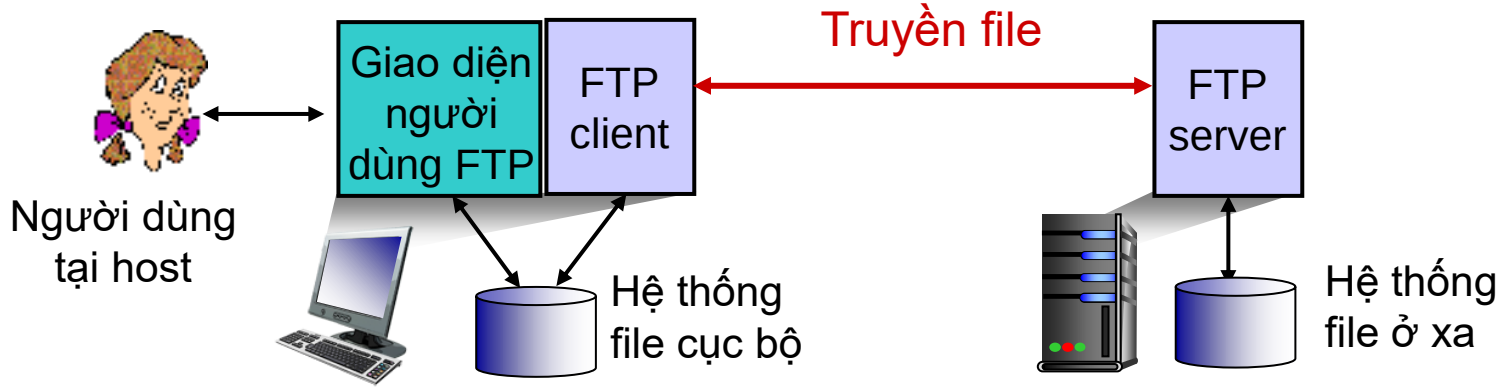
2.4 Thư điện tử
- SMTP, POP3, IMAP

2.5 DNS

2.6 Ứng dụng P2P

2.7 Lập trình socket với UDP và TCP

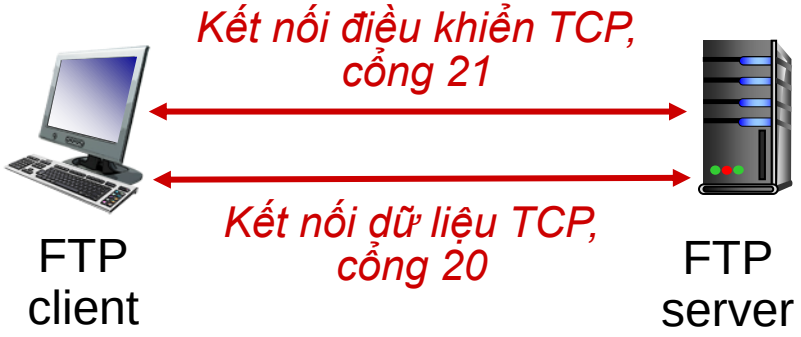
FTP: giao thức truyền file (file transfer protocol)



- Truyền file đến/từ host ở xa
- Mô hình client/server
  - *client:* phía khởi tạo việc truyền (đến/từ host ở xa)
  - *server:* host ở xa
- ftp: RFC 959
- ftp server: cổng 21

FTP: kết nối dữ liệu và kết nối điều khiển riêng biệt nhau

- FTP client tiếp xúc với FTP server tại cổng số 21, sử dụng TCP
- client được cấp phép qua kết nối điều khiển
- client duyệt thư mục ở xa, gửi lệnh qua kết nối điều khiển
- Khi server nhận lệnh truyền file, *server* mở kết nối TCP thứ 2 (kết nối dữ liệu) (cho file) *tới* client
- Sau khi truyền một file, server đóng kết nối dữ liệu.



- server mở một kết nối dữ liệu khác để truyền một file khác.
- Kết nối điều khiển: *"out of band"*
- FTP server duy trì "trạng thái": thư mục hiện hành, sự cấp phép trước đó.

Các lệnh và đáp ứng của FTP

Ví dụ các lệnh:

- Được gửi như các văn bản dạng mã ASCII qua kênh điều khiển
- **USER *username***
- **PASS *password***
- **LIST** trả về danh sách các file trong thự mục hiện hành
- **RETR filename** trích chọn (lấy) file
- **STOR filename** lưu (đặt) file vào host ở xa

Ví dụ các mã lệnh trả về

- Mã trạng thái và cụm từ trạng thái (như trong HTTP)
- **331 Username OK, password required**
- **125 data connection already open; transfer starting**
- **425 Can’t open data connection**
- **452 Error writing file**

Chương 2: Nội dung

2.1 Nguyên lý của ứng dụng mạng
- Kiến trúc của ứng dụng
- Các yêu cầu của ứng dụng

2.2 Web và HTTP

2.3 FTP

2.4 Thư điện tử
- SMTP, POP3, IMAP

2.5 DNS

2.6 Ứng dụng P2P

2.7 Lập trình socket với UDP và TCP

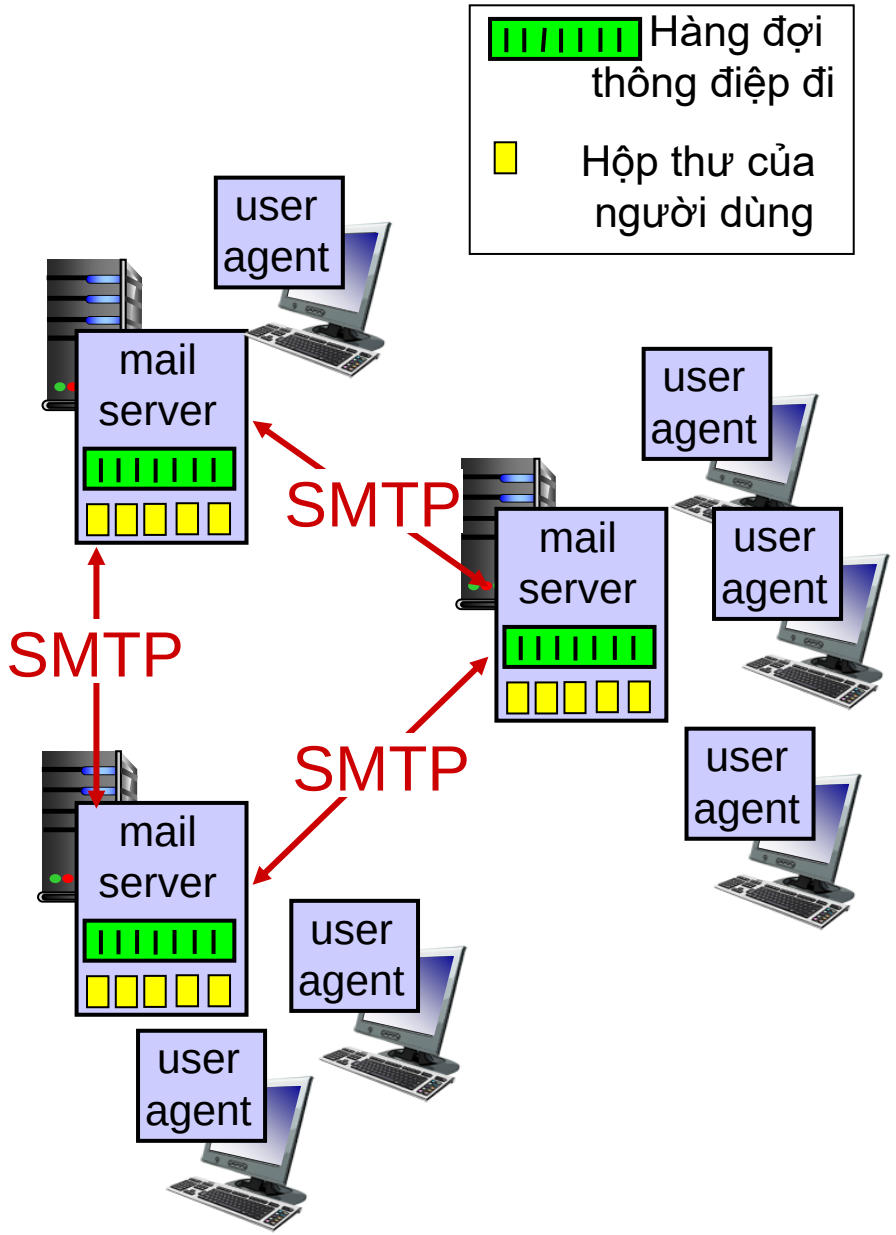
Thư điện tử

Ba thành phần chính:

- user agent
- mail server
- Giao thức truyền thư đơn giản (simple mail transfer protocol): SMTP

User Agent

- Còn được gọi là "mail reader"
- Soạn thảo, sửa, đọc các thông điệp thư
- Ví dụ: Outlook, Thunderbird, iPhone mail client
- Các thông điệp đi/đến được lưu trên server



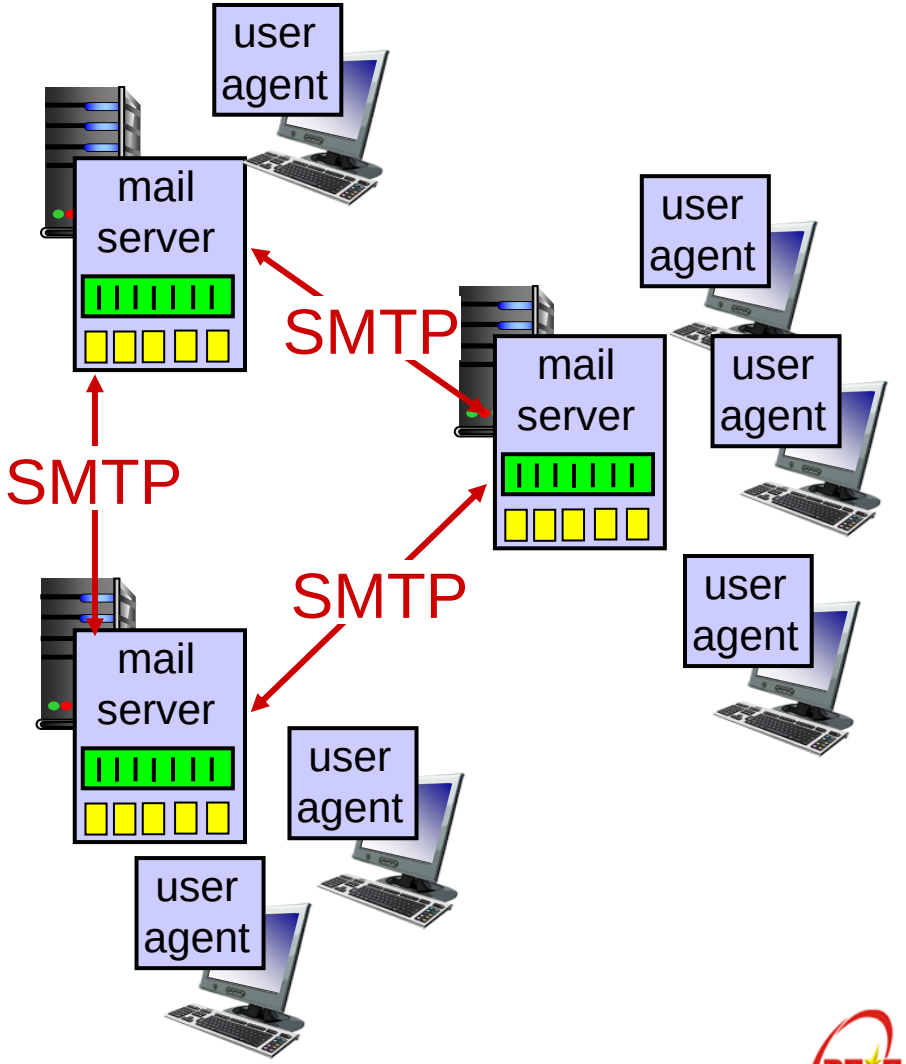
Thư điện tử: mail server

mail server:

- *Hộp thư (mailbox)* chứa các thông điệp thư đi đến người dùng
- *Hàng đợi thông điệp (message queue)* của các thông điệp thư đi ra ngoài (chuẩn bị được gửi đi)

Giao thức SMTP giữa các mail server thực hiện gửi các thông điệp thư

- client: gửi đến mail server
- server: mail server nhận



Thư điện tử: SMTP [RFC 5321]

- Sử dụng TCP để truyền tin cậy thông điệp thư điện tử từ client đến server, trên cổng số 25
- Truyền trực tiếp: từ server gửi tới server nhận
- Truyền theo 3 bước
  - Bắt tay (chào hỏi)
  - Truyền thông điệp
  - Đóng
- Tương tác lệnh/đáp ứng (giống như HTTP, FTP)
  - Lệnh: văn bản ASCII
  - Đáp ứng: mã trạng thái và các cụm từ trạng thái
- Các thông điệp phải ở dạng mã ASCII 7-bit

Tự thử nghiệm tương tác SMTP:

- **`telnet servername 25`**
- Thấy 220 trả lời từ server
- Nhập các lệnh HELO, MAIL FROM, RCPT TO, DATA, QUIT

Các lệnh trên cho phép gửi email mà không cần dùng email client (reader)

SMTP

- SMTP sử dụng kết nối bền vững
- SMTP yêu cầu thông điệp (phần tiêu đề & phần thân) ở dạng mã ASCII 7-bit
- SMTP server sử dụng `CRLF.CRLF` để xác định điểm kết thúc của thông điệp

So sánh với HTTP:

- HTTP: kéo
- SMTP: đẩy
- Cả hai đều sử dụng tương tác lệnh/đáp ứng, các mã trạng thái dạng bảng mã ASCII
- HTTP: mỗi đối tượng được đóng gói trong thông điệp đáp ứng của nó
- SMTP: nhiều đối tượng được gửi trong thông điệp nhiều phần

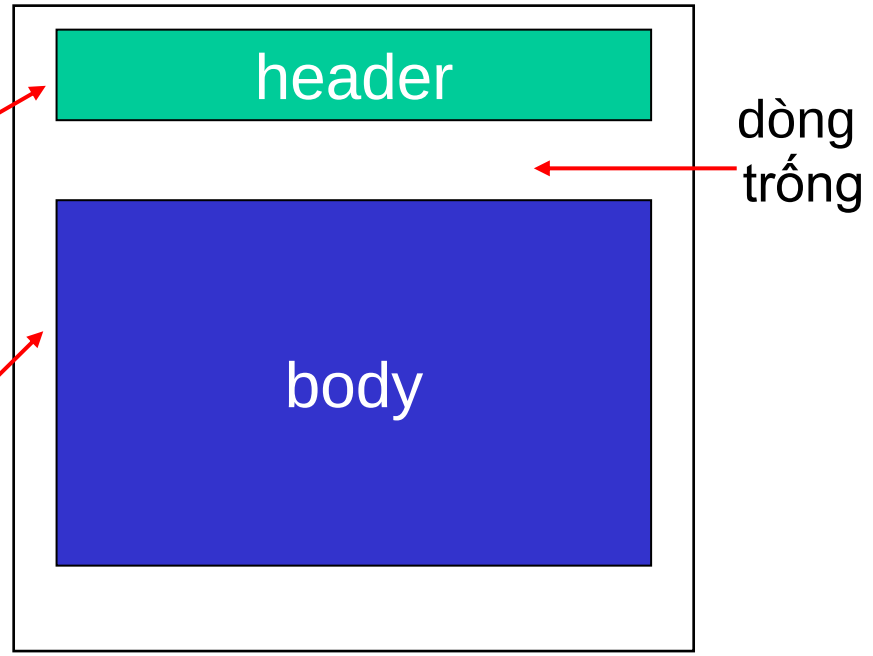
Định dạng thông điệp thư

SMTP: giao thức trao đổi các thông điệp thư

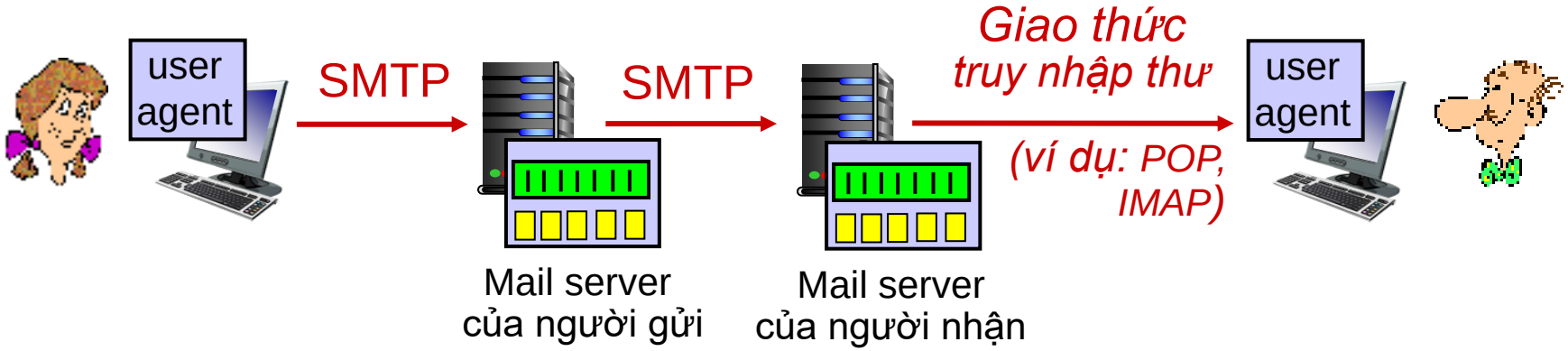
RFC 822: chuẩn cho định dạng thông điệp văn bản:

- Dòng tiêu đề, ví dụ:
  - To:
  - From:
  - Subject:

  Khác với các lệnh SMTP MAIL FROM, RCPT TO!
- Phần thân: "thông điệp"
  - Chỉ dùng các ký tự mã ASCII



Giao thức truy nhập thư



- SMTP: phân phối/lưu trữ thư tới/tại server của người nhận
- Giao thức truy nhập thư: trích xuất thư từ server
  - POP: Post Office Protocol [RFC 1939]: cấp phép, tải thư
  - IMAP: Internet Mail Access Protocol [RFC 3501]: có nhiều đặc tính hơn, bao gồm cả những thao tác với các thông điệp được lưu trữ trên server
  - HTTP: gmail, Hotmail, Yahoo! Mail,...

Giao thức POP3

Giai đoạn cấp phép

- Các lệnh của client:
  - **user:** khai báo tên người dùng
  - **pass:** mật khẩu
- Các đáp ứng của server
  - **+OK**
  - **-ERR**

Giai đoạn giao dịch, client:

- **list:** liệt kê các số thông điệp
- **retr:** trích xuất thông điệp theo số
- **dele:** xóa
- **quit**

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on
```

```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

POP3 (tiếp) và IMAP

POP3

- Ví dụ trước dùng POP3 với chế độ "tải và xóa"
  - Bob không thể đọc lại thư nếu thay đổi client
- POP3 với chế độ "tải và lưu giữ": sao các thông điệp lên các client khác nhau
- POP3 không giữ trạng thái của các phiên làm việc

IMAP

- Lưu giữ tất cả các thông điệp tại một nơi là server
- Cho phép người dùng tổ chức các thông điệp theo dạng các thư mục
- Lưu giữ trạng thái của người dùng qua các phiên làm việc:
  - Đặt tên thư mục và ánh xạ giữa các ID của thông điệp với tên thư mục

Chương 2: Nội dung

2.1 Nguyên lý của ứng dụng mạng
- Kiến trúc của ứng dụng
- Các yêu cầu của ứng dụng

2.2 Web và HTTP

2.3 FTP

2.4 Thư điện tử
- SMTP, POP3, IMAP

2.5 DNS

2.6 Ứng dụng P2P

2.7 Lập trình socket với UDP và TCP

DNS: Hệ thống tên miền (domain name system)

Con người: có nhiều định danh:

- CCCD, tên, số hộ chiếu

Các host và router trên Internet:

- Địa chỉ IP (32 bit) – được dùng để gán địa chỉ cho các gói tin
- "tên miền", ví dụ: www.yahoo.com – được con người sử dụng

Hỏi: Làm cách nào để ánh xạ giữa địa chỉ IP và tên miền, và ngược lại?

Hệ thống tên miền:

- *Cơ sở dữ liệu phân tán* được cài đặt phân cấp với nhiều *server tên miền*
- *Giao thức tầng ứng dụng:* để các host, các server tên miền truyền thông được thì cần phải *phân giải* các tên miền (diễn dịch địa chỉ/tên miền)
  - Chú ý: chức năng lõi của Internet, được cài đặt như giao thức tầng ứng dụng
  - Phức tạp tại "phần biên" của mạng

DNS: các dịch vụ và cấu trúc

Các dịch vụ của DNS

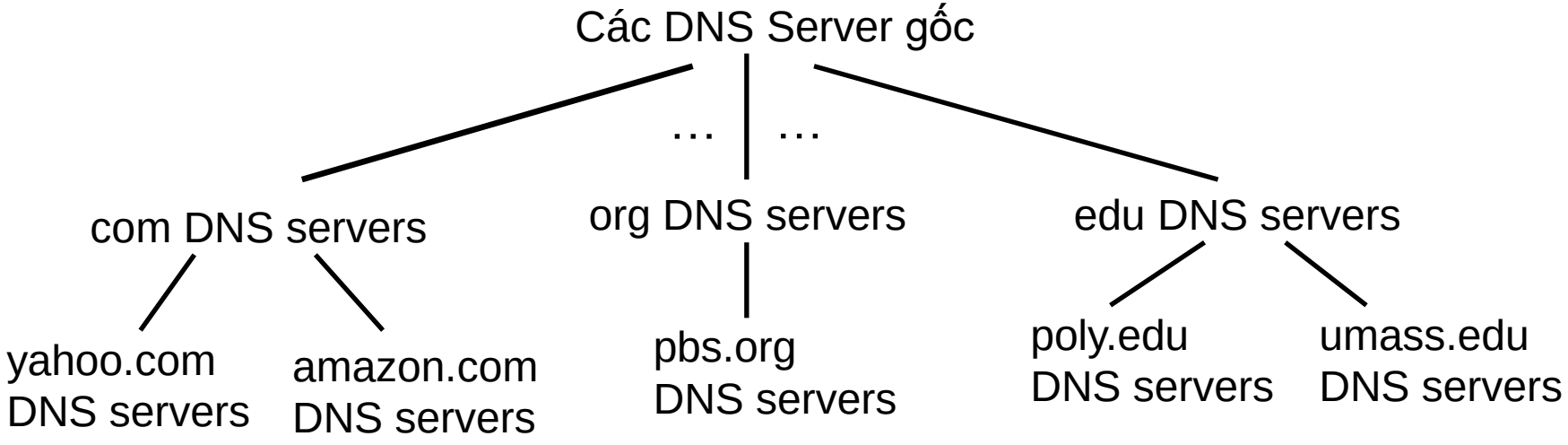
- Dịch tên host thành địa chỉ IP
- Bí danh của host
  - Các tên đúng chuẩn, các tên là bí danh
- Bí danh mail server
- Phân tán tải
  - Nhân rộng các máy chủ Web: nhiều địa chỉ IP tương ứng với một tên

Tại sao không tập trung hóa trong DNS?

- Một điểm chịu lỗi
- Lưu lượng
- Cơ sở dữ liệu tập trung ở xa
- Bảo trì

Trả lời: Không thể thực hiện với quy mô lớn!

DNS: cơ sở dữ liệu phân tán và phân cấp

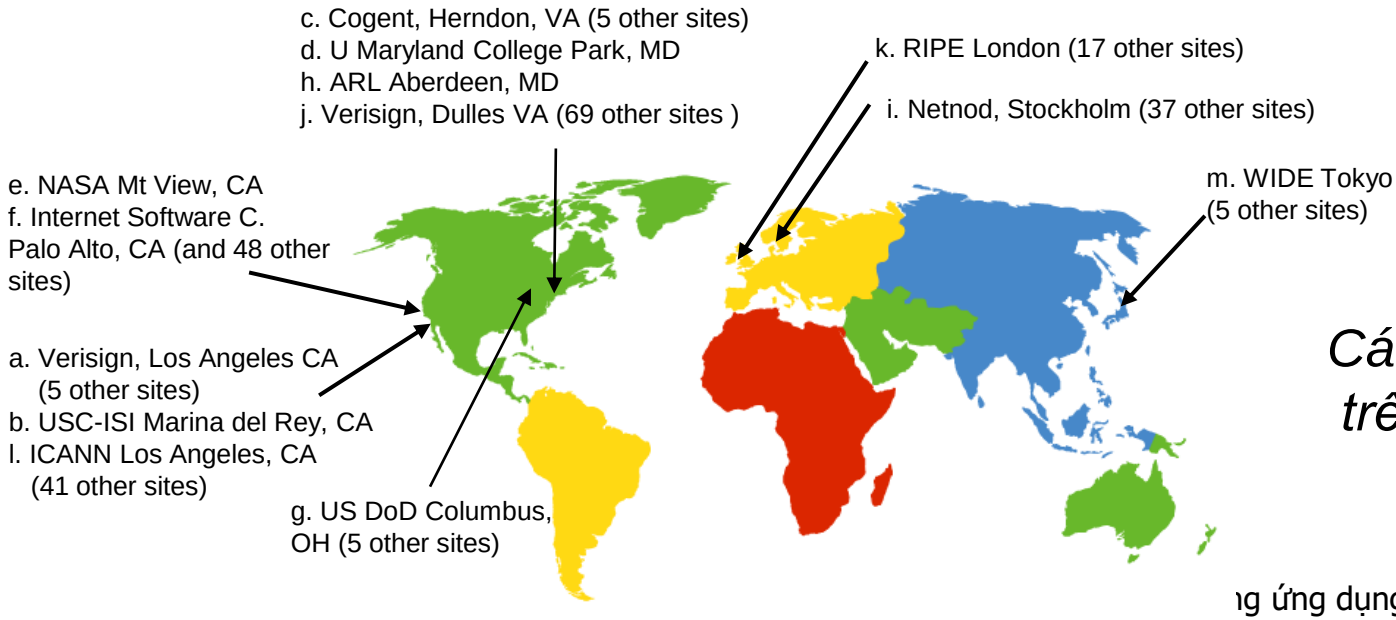


client muốn IP cho www.amazon.com:

- client truy vấn server gốc để tìm com DNS server
- client truy vấn .com DNS server để lấy amazon.com DNS server
- client truy vấn amazon.com DNS server để lấy địa chỉ IP của www.amazon.com

DNS: Các server tên gốc

- Được tiếp xúc bởi server tên cục bộ mà server này không có khả năng phân giải tên
- Server tên gốc:
  - Liên lạc với server tên có thẩm quyền nếu việc ánh xạ tên không được xác định
  - Lấy ánh xạ
  - Trả lại ánh xạ cho server tên cục bộ



Các server tên gốc trên toàn thế giới

Các server TLD và server có thẩm quyền

Các server top-level domain (TLD):

- Chịu trách nhiệm cho tên miền com, org, net, edu, aero, jobs, museums, và tất cả các tên miền cấp cao nhất thuộc quốc gia như: uk, fr, ca, jp
- Network Solutions duy trì các server cho .com TLD
- Educause cho .edu TLD

Các server DNS có thẩm quyền:

- Các DNS server của tổ chức, cung cấp các tên host có thẩm quyền cho các ánh xạ IP với các host của tổ chức (ví dụ web và mail).
- Có thể được duy trì bởi các tổ chức hoặc nhà cung cấp dịch vụ

Server tên DNS cục bộ

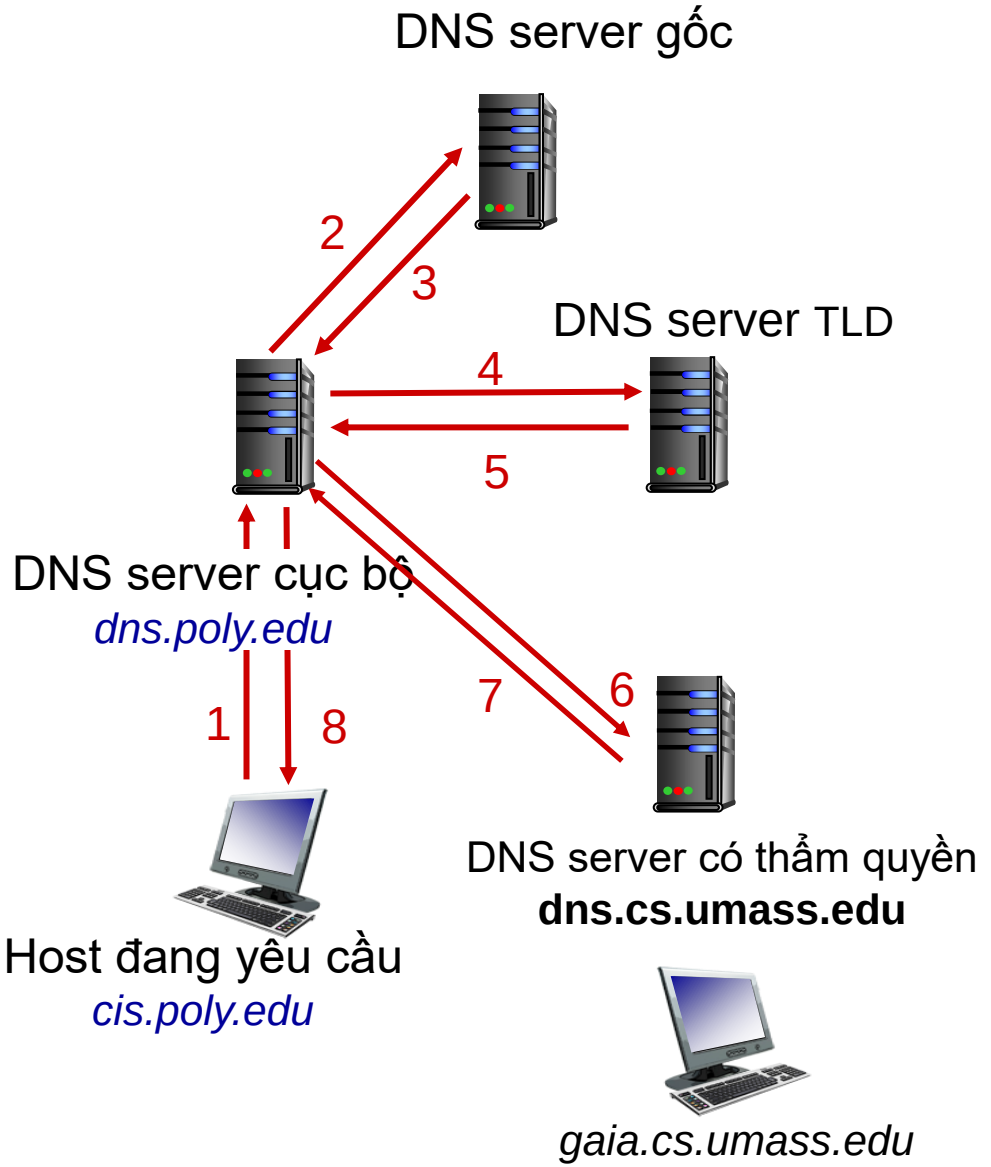
- Không hoàn toàn theo cấu trúc phân cấp
- Mỗi ISP (ISP khu dân cư, công ty, trường học) có một server cục bộ
  - Cũng được gọi là "server tên mặc định"
- Khi một host tạo một truy vấn DNS, truy vấn này sẽ được gửi tới server DNS cục bộ
  - Có một vùng đệm cục bộ dịch chuyển cặp tên-địa chỉ gần nhất (tuy nhiên vẫn có thể đã bị quá hạn, chưa được cập nhật!)
  - Hoạt động giống như proxy, chuyển tiếp truy vấn vào hệ thống phân cấp

Ví dụ phân giải tên DNS

- Host tại cis.poly.edu muốn lấy địa chỉ IP của gaia.cs.umass.edu

Truy vấn lặp:

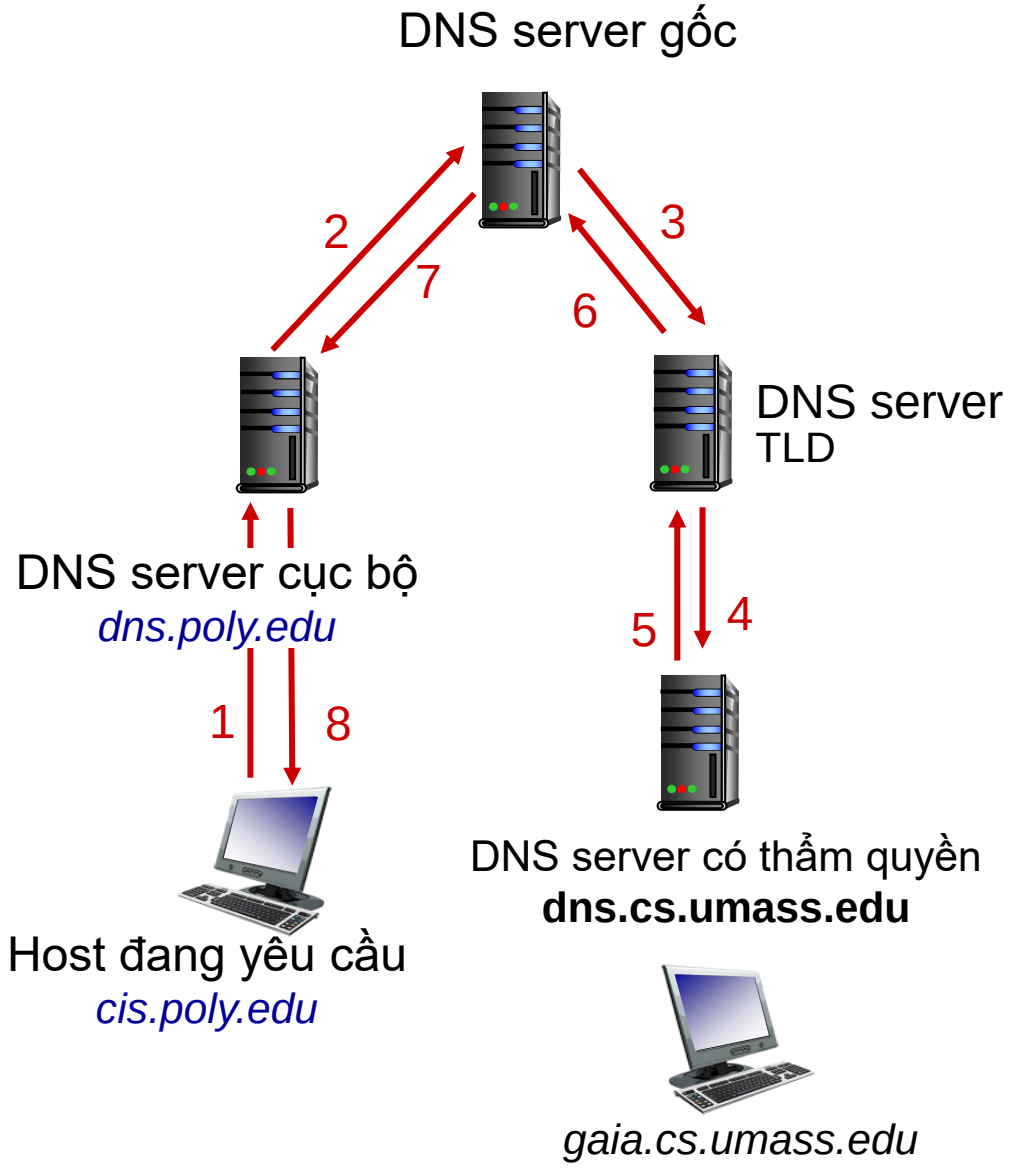
- Server được hỏi sẽ trả lời với tên của server sẽ có thể hỏi được tiếp
- "Tôi không biết tên đó, nhưng có thể hỏi server này"



Ví dụ phân giải tên DNS

Truy vấn đệ quy:

- Đặt trách nhiệm phân giải tên lên server tên được hỏi
- Tải nặng tại các cấp cao hơn trong hệ thống phân cấp?



DNS: caching và cập nhật các bản ghi

- Khi server tên học cách ánh xạ, nó sẽ *cache* ánh xạ
  - Mục cache sẽ bị quá hạn (bị xóa) sau một vài lần (TTL)
  - Các server TLD điển hình thường cache trong các server tên cục bộ
    - Do đó, các server tên gốc sẽ không thường xuyên được truy cập
- Các mục cache có thể bị *quá hạn*
  - Nếu host thay đổi địa chỉ IP, thì Internet sẽ có thể không biết được cho đến khi TTL hết hạn
- Cơ chế cập nhật, thông báo được đề xuất bởi chuẩn IETF
  - RFC 2136

Bản ghi DNS (DNS records)

DNS: cơ sở dữ liệu phân tán lưu trữ các bản ghi nguồn (resource records - RR)

Định dạng RR: `(name, value, type, ttl)`

type=A

- **name** là tên máy
- **Value** là địa chỉ IP

type=NS

- **name** là tên miền (ví dụ: foo.com)
- **value** là tên host của server có thẩm quyền cho tên miền này

type=CNAME

- **name** là tên bí danh của tên "chuẩn" (tên thật)
- `www.ibm.com` tên thật là `servereast.backup2.ibm.com`
- **value** là tên chuẩn

type=MX

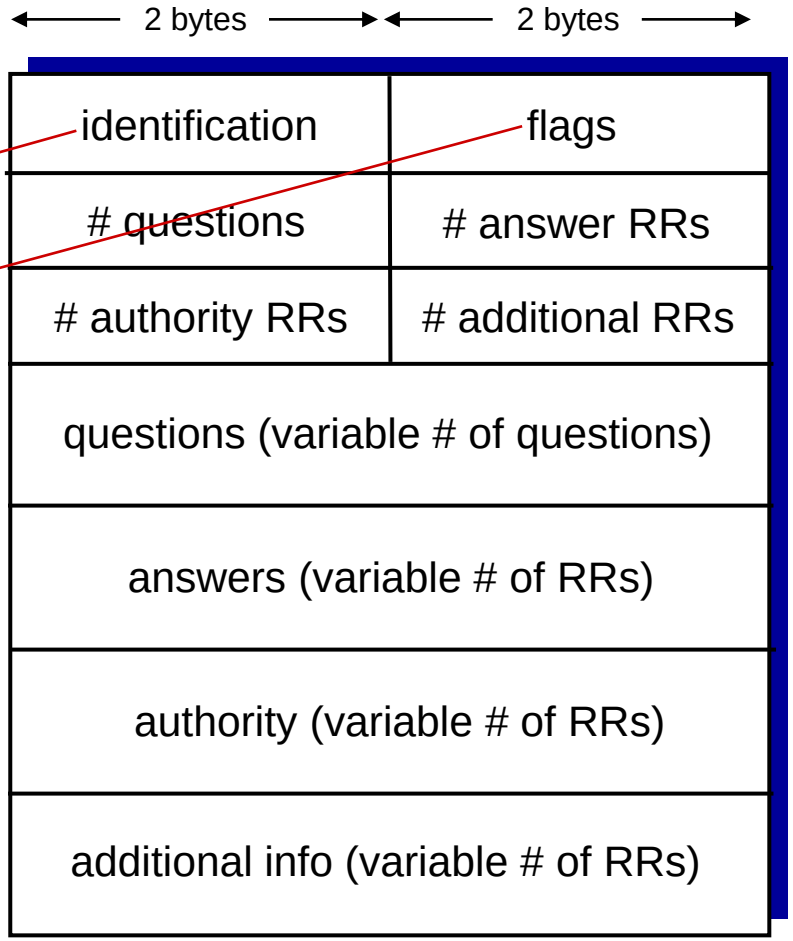
- **value** là tên của mail server được kết hợp với **name**

Giao thức, thông điệp DNS

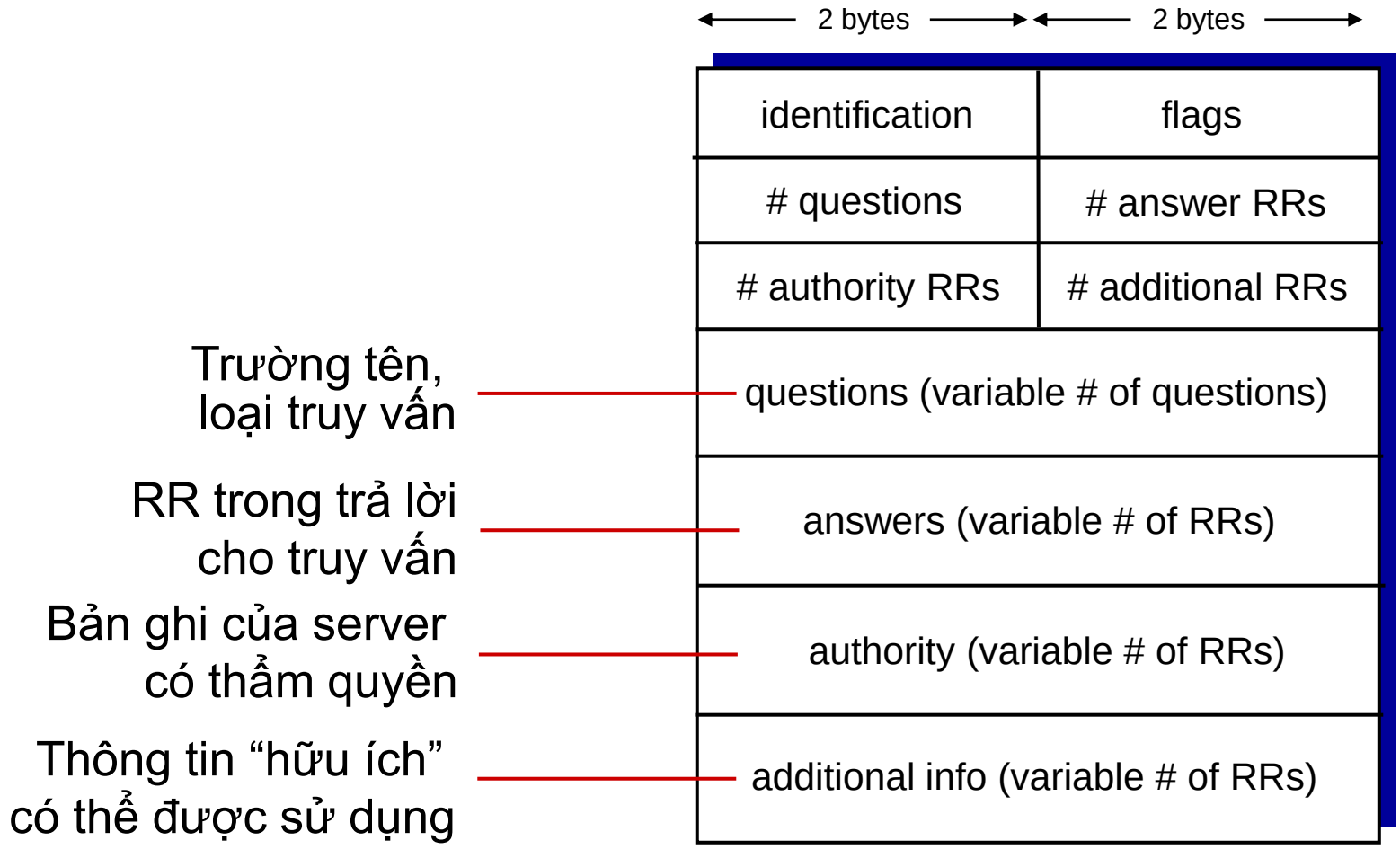
- Các thông điệp *truy vấn* và *trả lời* đều có cùng *định dạng thông điệp*

Tiêu đề thông điệp

- identification: 16 bit cho truy vấn/trả lời
- flags:
  - Truy vấn hoặc trả lời
  - Đệ quy chờ
  - Đệ quy sẵn sàng
  - Trả lời có thẩm quyền



Giao thức, thông điệp DNS



Chèn thêm bản ghi vào trong DNS

- Ví dụ: Tạo mới "Network Utopia"
- Đăng ký tên miền networkuptopia.com tại *DNS registrar* (Ví dụ: Network Solutions)
  - Cung cấp tên, địa chỉ IP của server tên có thẩm quyền (sơ cấp và thứ cấp) của bạn
  - Registrar sẽ chèn hai bản ghi RR vào trong .com TLD server:
    (networkutopia.com, dns1.networkutopia.com, NS)
    (dns1.networkutopia.com, 212.212.212.1, A)
- Tạo bản ghi loại A vào trong server có thẩm quyền cho www.networkuptopia.com; bản ghi loại MX record cho networkutopia.com

Tấn công DNS

Tấn công DDoS

- Tấn công server gốc với lưu lượng lớn
  - Không thành công cho đến nay
  - Lọc lưu lượng
  - DNS server cục bộ cache IP của TLD server, cho phép bỏ qua server gốc
  - Tấn công TLD servers
  - Tiềm tàng nhiều nguy hiểm hơn

Chuyển hướng tấn công

- Man-in-middle
  - Truy vấn đánh chặn
- Đầu độc DNS
  - Gửi trả lời giả mạo đến DNS server, mà các server này có thể cache lại

Khai thác DNS cho DDoS

- Gửi truy vấn với địa chỉ nguồn giả mạo: IP đích
- Yêu cầu khuếch đại

Chương 2: Nội dung

2.1 Nguyên lý của ứng dụng mạng
- Kiến trúc của ứng dụng
- Các yêu cầu của ứng dụng

2.2 Web và HTTP

2.3 FTP

2.4 Thư điện tử
- SMTP, POP3, IMAP

2.5 DNS

2.6 Ứng dụng P2P

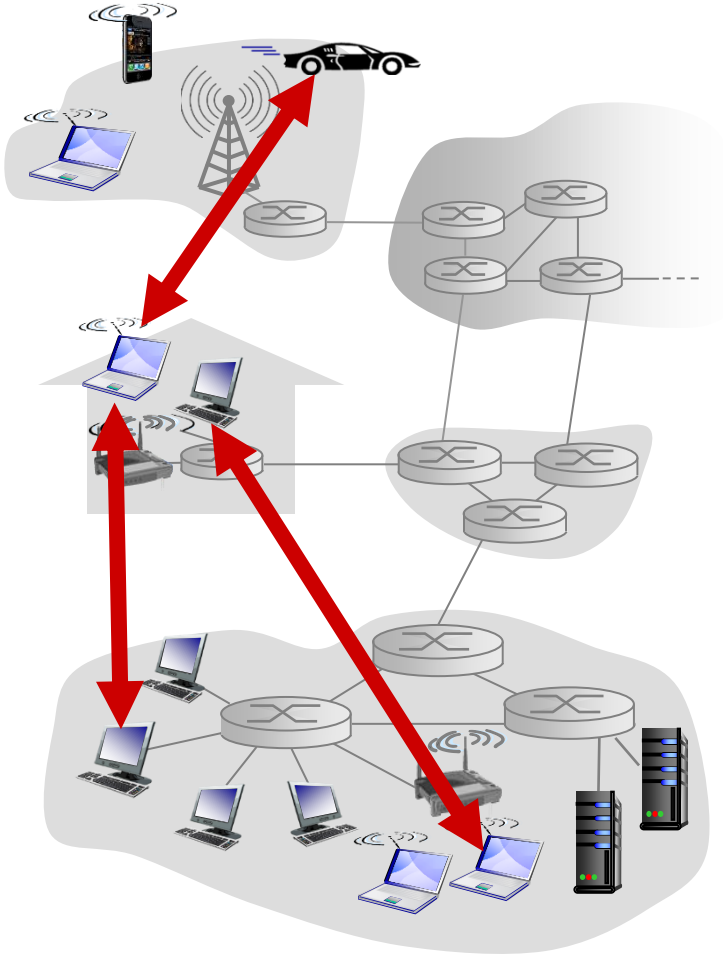
2.7 Lập trình socket với UDP và TCP

Kiến trúc mạng P2P thuần túy

- *Không* cần phải có server luôn hoạt động
- Các hệ thống cuối tùy ý kết nối trực tiếp
- Các peer (các nút mạng) không cần kết nối liên tục vào hệ thống mạng và có thể thay đổi địa chỉ IP.

Ví dụ:

- Chia sẻ file (BitTorrent)
- Streaming (KanKan)
- VoIP (Skype)

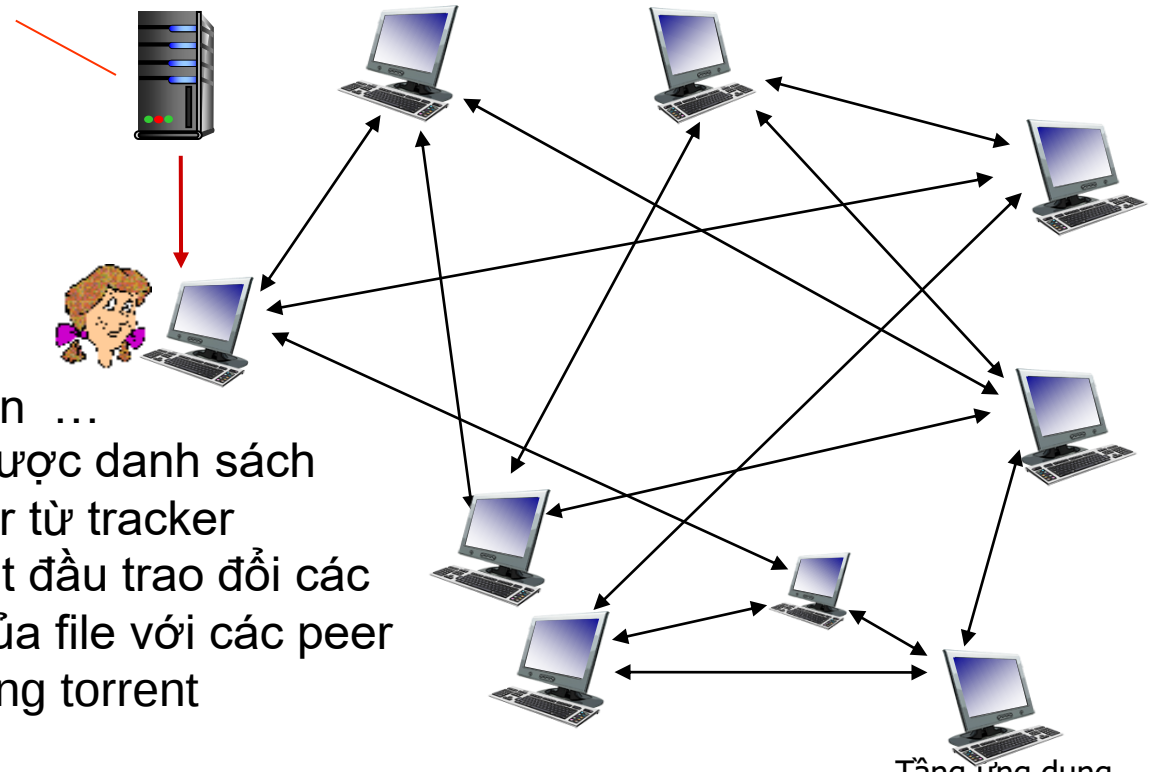


Truyền file P2P: BitTorrent

- File được chia thành các chunk (phần) có kích thước là 256Kb
- Các peer trong torrent gửi/nhận các chunk của file

tracker: kiểm tra các peer đang tham gia trong torrent

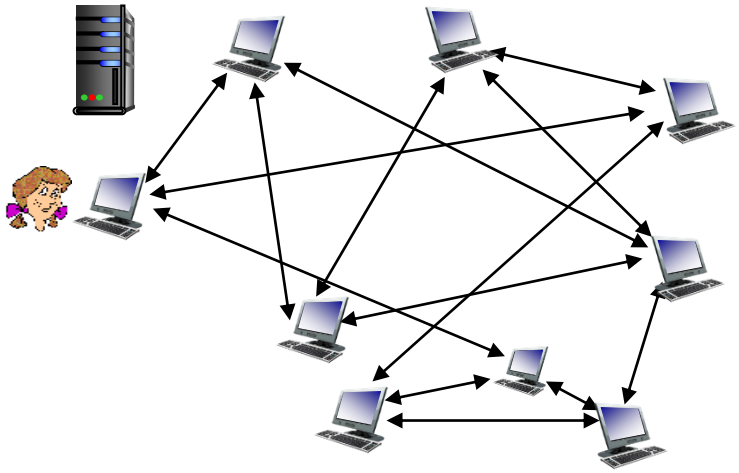
torrent: nhóm các peer trao đổi các chunk của một file



Alice đến …
… lấy được danh sách các peer từ tracker
… và bắt đầu trao đổi các chunk của file với các peer khác trong torrent

Truyền file P2P: BitTorrent

- peer tham gia vào torrent:
  - Không có các chunk, nhưng sẽ tích lũy chúng qua thời gian từ các peer khác
  - Đăng ký với tracker để nhận được danh sách các peer, kết nối với tập con của các peer (“các nút lân cận”)



- Trong khi download, peer sẽ upload các chunk tới các peer khác
- peer có thể thay đổi các peer mà nó sẽ trao đổi các chunk
- *churn:* các peer có thể đến hoặc đi
- Khi peer có được toàn bộ file, nó có thể (ích kỷ) rời đi hoặc (vị tha) ở lại trong torrent

BitTorrent: yêu cầu lấy, gửi các chunk của file

Yêu cầu lấy chunk:

- Tại bất kỳ thời điểm nào, các peer khác nhau đều có các tập con các chunk khác nhau của file.
- Theo định kỳ, Alice sẽ hỏi mỗi peer về danh sách các chunk mà họ có
- Alice sẽ yêu cầu các chunk còn thiếu từ các peer, phần hiếm nhất trước

Gửi chunk: tit-for-tat

- Alice gửi các chunk tới 4 peer hiện đang gửi chunk của cô ấy với *tốc độ cao nhất*
  - Các peer khác đang bị chặn bởi Alice (không nhận được chunk từ cô ấy)
  - Đánh giá lại top 4 sau mỗi 10 giây
- Cứ mỗi 30 giây: chọn các peer khác một cách ngẫu nhiên, và bắt đầu gửi chunk
  - "Tối ưu hóa không chặn" peer này
  - Peer được chọn mới sẽ được gia nhập vào top 4

Chương 2: Nội dung

2.1 Nguyên lý của ứng dụng mạng
- Kiến trúc của ứng dụng
- Các yêu cầu của ứng dụng

2.2 Web và HTTP

2.3 FTP

2.4 Thư điện tử
- SMTP, POP3, IMAP

2.5 DNS

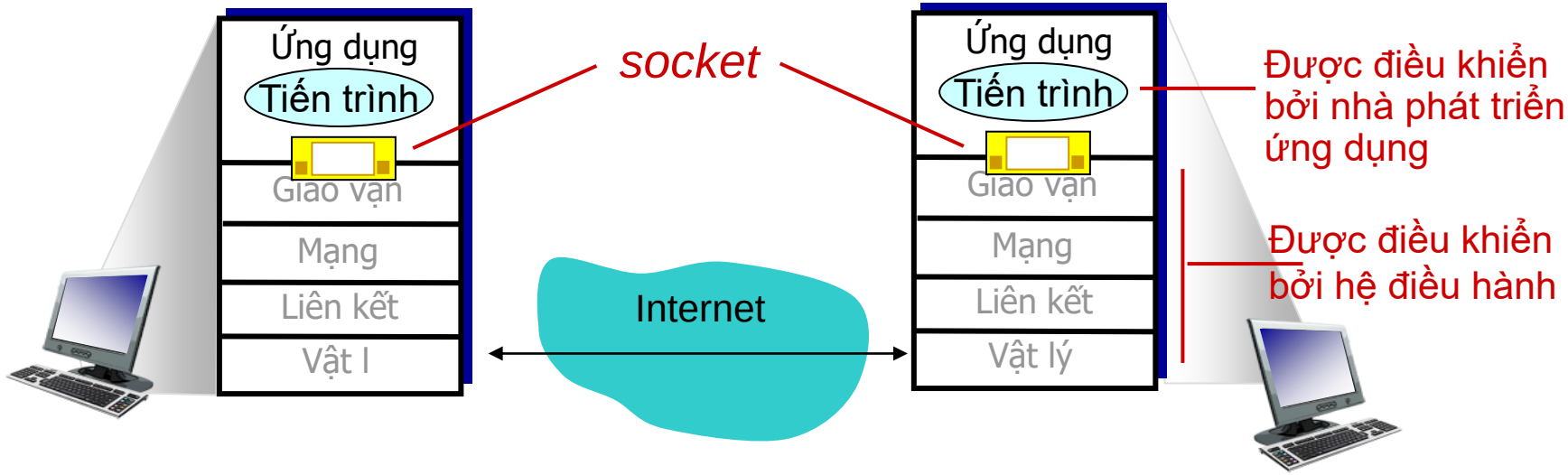
2.6 Ứng dụng P2P

2.7 Lập trình socket với UDP và TCP

Lập trình Socket

Mục đích: hiểu được cách xây dựng ứng dụng truyền thông client/server dùng socket

socket: là cánh cửa giữa các tiến trình ứng dụng và giao thức giao vận end-to-end



Lập trình Socket

Hai loại socket cho hai dịch vụ tầng giao vận:

- *UDP:* truyền các gói tin không tin cậy
- *TCP:* truyền tin cậy, truyền dòng byte có hướng

Ví dụ ứng dụng:

1. Client đọc vào một dòng ký tự (dữ liệu) từ bàn phím và gửi dữ liệu đến server.
2. Server nhận dữ liệu và chuyển các ký tự thành dạng ký tự viết hoa.
3. Server gửi dữ liệu đã được chuyển thành dạng viết hoa về cho client.
4. Client nhận dữ liệu và hiển thị dòng ký tự lên màn hình.

Lập trình Socket *với UDP*

UDP: không có "kết nối" giữa client & server

- Không bắt tay trước khi gửi dữ liệu
- Bên gửi gắn địa chỉ IP và số hiệu cổng đích vào trong mỗi gói tin
- Bên nhận sẽ trích địa chỉ IP và số hiệu cổng của bên gửi từ gói tin nhận được

UDP: dữ liệu được truyền có thể bị mất hoặc không đúng trình tự khi nhận

Quan điểm ứng dụng:

- UDP cung cấp truyền không tin cậy theo các nhóm byte ("các gói tin") giữa client và server

Lập trình socket *với TCP*

client phải tiếp xúc với server

- Tiến trình server phải chạy trước
- server phải tạo socket (cửa) để đón client tiếp xúc

client tiếp xúc với server bằng cách:

- Tạo TCP socket, xác định địa chỉ IP, số hiệu cổng của tiến trình server
- *Khi client tạo socket:* TCP client sẽ thiết lập kết nối tới TCP server

- Khi được tiếp xúc bởi client, *TCP server sẽ tạo socket mới* cho tiến trình server để truyền thông với client
  - Cho phép server "nói chuyện" với nhiều client
  - Các số hiệu cổng nguồn được dùng để phân biệt các client (xem thêm trong Chương 3)

Quan điểm ứng dụng:

TCP cung cấp truyền tin cậy, truyền dòng byte theo đúng trình tự giữa client và server.

Xây dựng một Web server đơn giản

- Quản lý một yêu cầu HTTP
- Chấp nhận yêu cầu
- Phân tích cú pháp phần tiêu đề (header)
- Lấy file được yêu cầu từ hệ thống file của server
- Tạo thông điệp đáp ứng HTTP:
  - các dòng header + file
- Gửi đáp ứng đến client

- Sau khi tạo server, có thể yêu cầu file dùng trình duyệt (ví dụ: IE)
- Xem giáo trình để biết thêm chi tiết

Chương 2: Tổng kết

Trình bày các vấn đề liên quan đến ứng dụng mạng!

- Kiến trúc của ứng dụng
  - client-server
  - P2P
- Các yêu cầu dịch vụ của ứng dụng:
  - Truyền tin cậy, băng thông, trễ
- Mô hình dịch vụ giao vận của Internet
  - Hướng kết nối, truyền tin cậy: TCP
  - Truyền không tin cậy, truyền gói tin: UDP
- Các giao thức cụ thể:
  - HTTP
  - FTP
  - SMTP, POP, IMAP
  - DNS
  - P2P: BitTorrent, DHT
- Lập trình socket: TCP socket, UDP socket